

Agenda:

↳ Problem solving on trees.

Recursion:

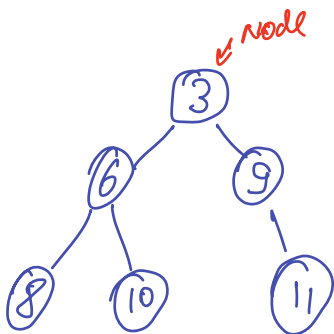
ASS: { decide, what your function do, assume it does }

main logic: { solving problems with subproblems }

Base Condition: { when code stops }

Q1)

Given Binary tree and an integer K , find out whether a node with value K is present or not.



$K=6? \rightarrow$

$K=9 \checkmark$

$K=15 \times$

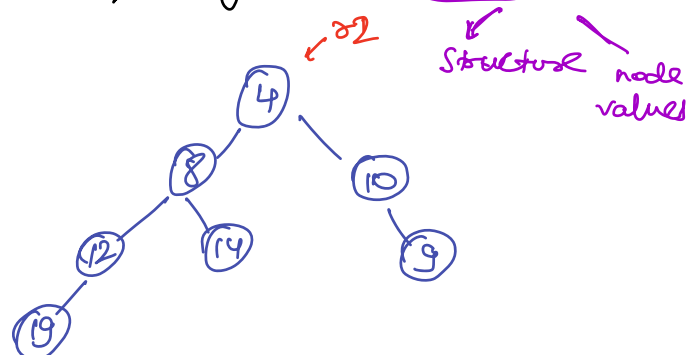
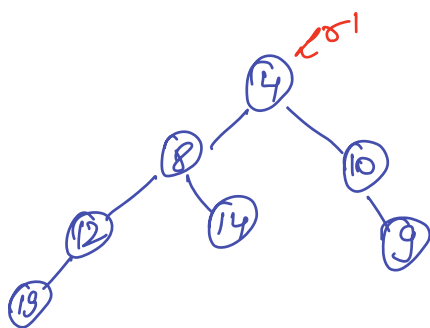
Preorder: NLR

→ Do traversal and check whether Node with K is Present or not.

↳ Any traversal would do, but Prefer Preorder.

Q2)

Given 2 BT check if they are Similar.



bool checkSim(Node s1, Node s2) { ASS: Check if
if (s1 == null && s2 == null) { return true; }
if (s1 == null || s2 == null) { return false; }
if (s1.data != s2.data) {
return false;
}

boolean leftSimilar = checkSim(s1.left, s2.left); main logic:

boolean rightSimilar = checkSim(s1.right, s2.right);

if (leftSimilar == true && rightSimilar == true) {
base case:

return true;
}
return false;

a) ① Check for root ② then ask for left & right subtree.

b) ① Ask for left & right subtree ② Check for root.

↳ both are correct, but prefer 1st



```

if (n1 == null && n2 != null) ||
    (n1 != null && n2 == null)) {
    // stop
    return false;
}
  
```

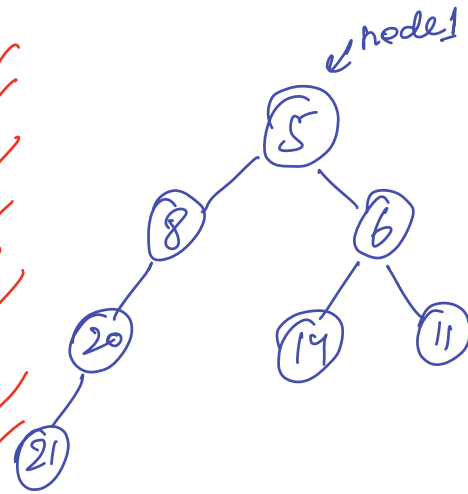
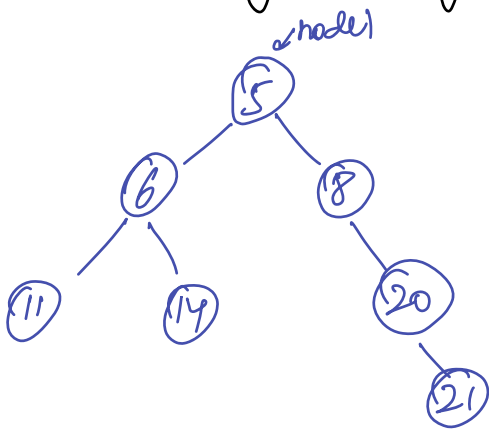
```

if (r1 == null && r2 == null) {
    // stop
    return true;
}
  
```

Time: $O(N)$

Space: $O(h) \rightarrow$ internal stack

Q3) Inverting binary tree



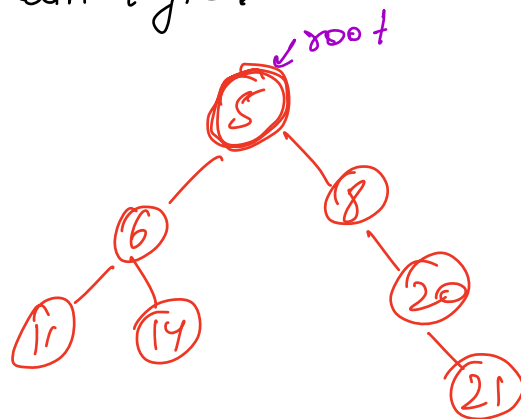
```
void invertBT(Node root) {  
    if (root == null) {  
        return;  
    }  
}
```

Ass: Given root node,
Invert binary tree.

Main logic:

```
invertBT(root.left);  
invertBT(root.right);
```

```
{  
    Node temp = root.left;  
    root.left = root.right;  
    root.right = temp;  
}
```



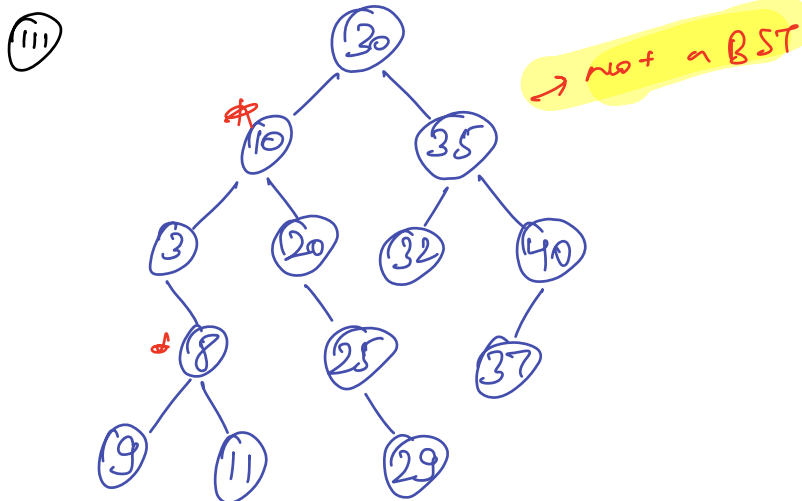
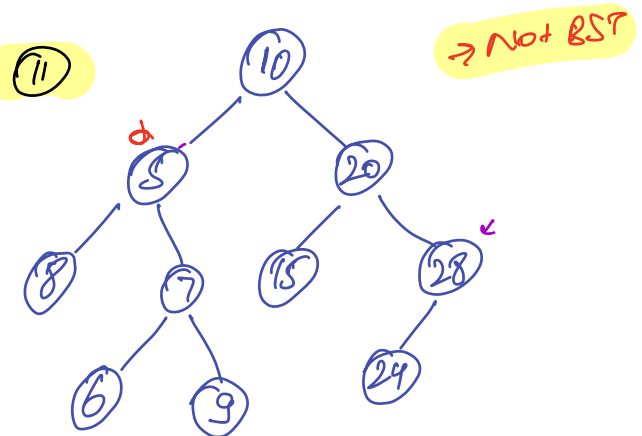
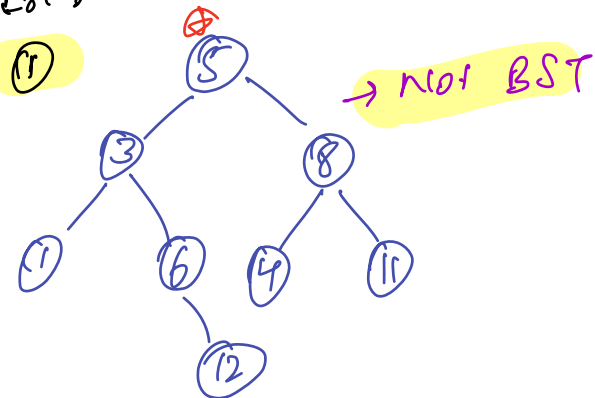
Base Case:

```
if (root == null) {  
    return;  
}
```

BST: Binary Search tree

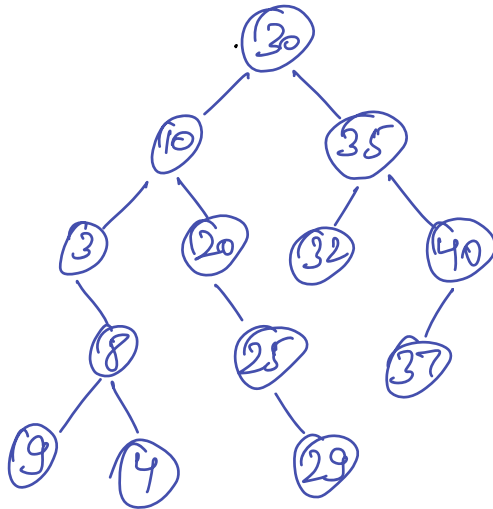
for all nodes: All the data of LST < Node data < All the data of RST

Ex:

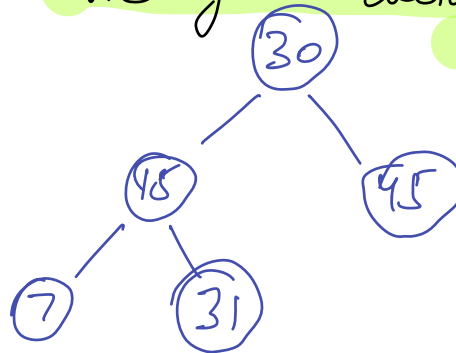


① for all nodes: All the data of LST < Node data < All the data of RST

② for all nodes: max of LST < node data < min RST

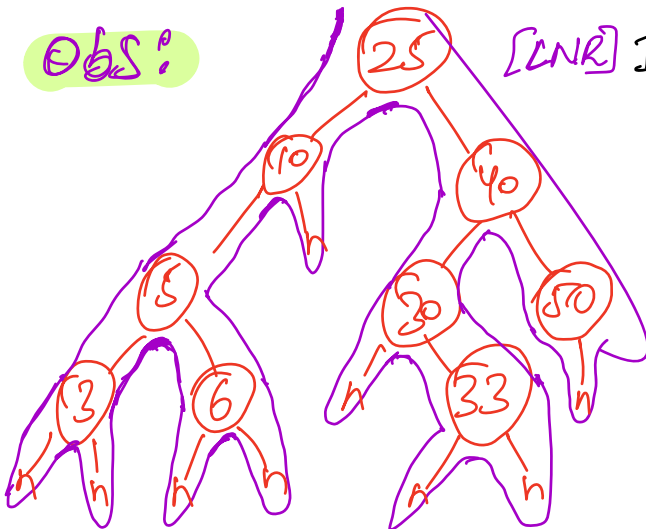


misconception: Don't just compare the left child and the right to decide about BST.



Obs:

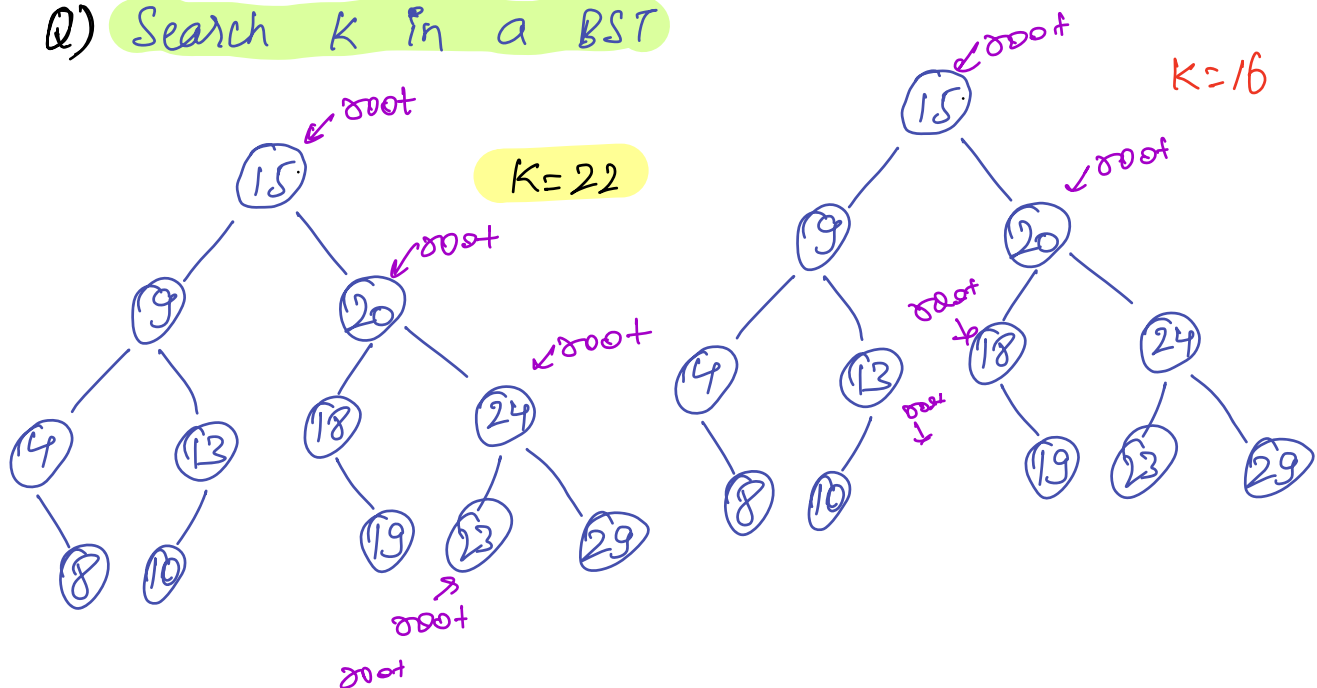
[LNR] Inorder: 3 5 6 10 25 30 33 40 50



for any BST, inorder will be sorted.

BT → Inorder → Sorted or not

Q) Search K in a BST



Pseudo code :

```
boolean Search(Node root, int K){
```

```
    while (root != null){
```

```
        if (root.data == K){
```

```
            return true;
```

```
        } else if (root.data > K){
```

```
            root = root.left;
```

```
        } else { // root.data < K
```

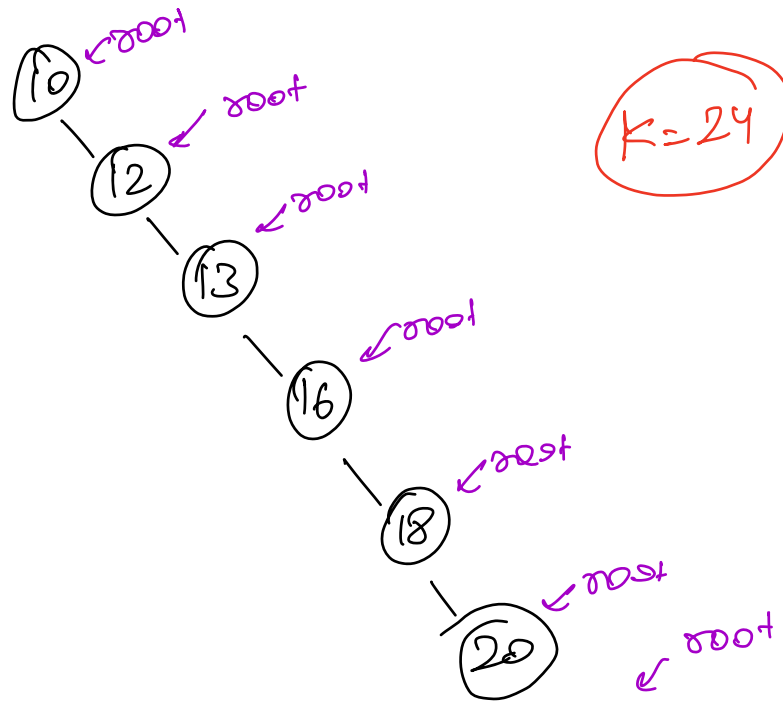
```
            root = root.right;
```

```
        }
```

```
    return false;
```

```
}
```

TC: $O(n) \rightarrow$ (worst case) $SC: O(1)$



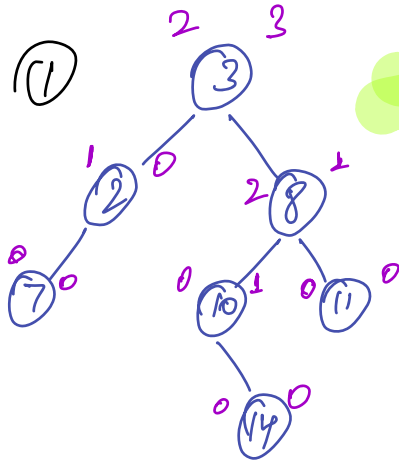
$\rightarrow$ whatsapp: 8076179594

Balanced binary tree :

height $\rightarrow$ nodes

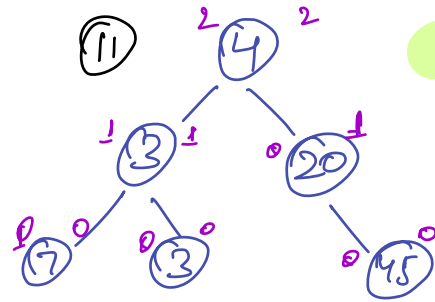
for all nodes: $\text{abs}(\text{height}[\text{LST}] - \text{height}[\text{RST}]) \leq 1$

Ex: (I)



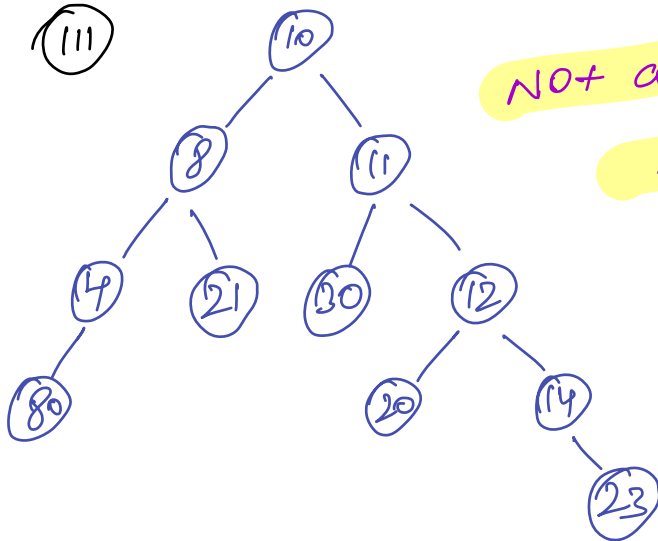
yes

(II)



yes

(III)



Not a balanced

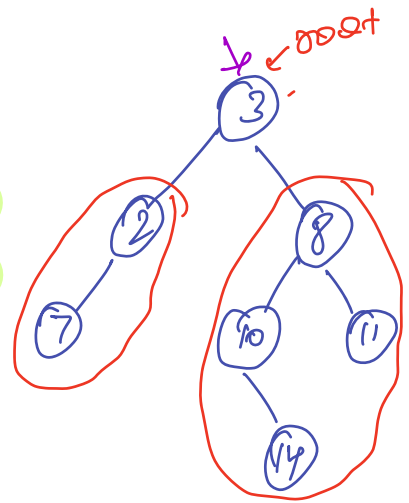
BT

boolean isbalanced = true;

```

int height (node root) {
    if (root == null) {
        return 0;
    }
    int lh = height (root->left);
    int rh = height (root->right);
    if (abs(lh - rh) > 1) {
        isbalanced = false;
    }
    return max(lh, rh) + 1;
}

```



* Doubt Session:

void invertBT(Node root) {

① if (node == null) {
 return;
}

② invertBT(root.left);

③ invertBT(root.right);

④ {
 Node temp = root.left;
 root.left = root.right;
 root.right = temp;
}

}

